

Wollo University
Department of Health Informatics
Internet Programming I

Instructor Name :- Daniel Bekele

Chapter 1: The Fundamentals of Internet Programming

1.1 Internet history, Uses and Services

The Internet is a vast, global network of interconnected computers and devices that provides the infrastructure for various services, while Internet services are the specific applications and tools that run on this infrastructure, allowing users to perform functions like browsing the web, sending emails, streaming video, and using social media. These services leverage the internet to share information, provide entertainment, and facilitate communication and commerce

Internet is:-

- **A Network of Networks:**

The Internet connects millions of end systems (like laptops and smartphones), routers, and communication links, creating a global, interconnected digital system.

- **An Infrastructure:**

It is the underlying hardware and protocols (set of rules) that allow devices to exchange data.

Internet Services are:

- **Applications and Tools:**

These are the programs and features that utilize the Internet's connectivity to provide useful functions.

- Common examples include:

- **World Wide Web (WWW):** Accessing websites and information through web browsers.
- **Email:** Sending and receiving messages.
- **Social Media:** Platforms for connecting with others.

- **Online Shopping (E-commerce):** Buying and selling goods and services online.
- **Streaming Platforms:** Watching videos and listening to music.
- **Online Learning (E-Learning):** Participating in educational activities.

1.2 The World Wide Web (WWW)

WWW stands for World Wide Web.

- It is a system of **interlinked documents and resources** on the internet, accessed through web browsers (like Chrome, Firefox, or Edge). These documents are connected using **hyperlinks** and identified by **Uniform Resource Locators (URLs)**.
- **Key points about WWW:**
 - **Developed by Tim Berners-Lee** in 1989–1991.
 - Uses **HTTP (Hypertext Transfer Protocol)** to transfer data.
 - Built on top of the internet (the internet is the infrastructure; the web is a service that runs on it).
 - Allows access to text, images, videos, and interactive content.
- **Key characteristics:**
 - **System of interconnected information:** It's a collection of documents, images, videos, and other resources that are linked together.
 - **Accessed through the Internet:** The World Wide Web uses the Internet as its underlying network to transfer data.
 - **Distinguished from the Internet:** While often used interchangeably, the Internet is the broader network, whereas the Web is the collection of websites and webpages.
 - **Requires a browser:** A web browser is a software application used to access and view content on the World Wide Web.
 - **Part of a URL:** "www" used to be a standard part of a website's address (e.g., **www.wu.edu.et**), but is often omitted today as browsers and web servers can handle it.

1.3 Client-Server Architectural Overview of WWW

The WWW uses a client-server model where client programs (like web browsers) request data or services from a server, which then processes the request and sends a response back. This interaction follows a request-response cycle, often using the **HTTP protocol**, and involves components like **DNS** to find server addresses, ports for specific communication channels, and APIs for defined interactions. The server stores and provides the requested resources, such as web pages or application data, enabling users to access information and services over the internet.

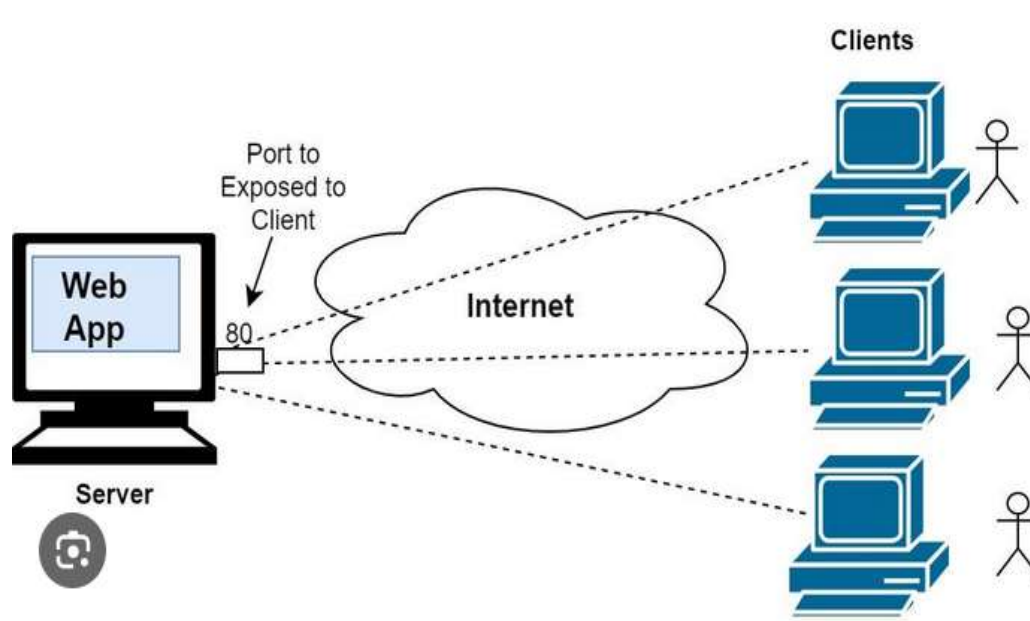


Figure 1.1. Simple Client-Server Architecture

Client-Server Model Explained

- **Client:**

A software program or device (like a web browser, smartphone, or other application) that initiates a request to a server for a service or data.

- **Server:**

A powerful computer or system designed to provide services, resources, and data to clients. It "listens" for incoming client requests, processes them, and sends back a response.

The World Wide Web works

1. **Request Initiation:** A user enters a web address (URL) or clicks a link in their web browser (the client).
2. **DNS Lookup:** The browser uses the Domain Name System (DNS) to translate the human-readable domain name (e.g **www.wu.edu.et**) into an IP address, which is a unique identifier for the server on the network.
3. **HTTP Request:** The browser then sends an **HTTP** request to the server's IP address, specifying the desired resource (e.g., a webpage). This request is sent over a specific port, often port 80 for HTTP or 443 for HTTPS.
4. **Server Processing:** The web server receives the request, processes it, and retrieves the requested data from its files or databases.
5. **HTTP Response:** The server sends an HTTP response back to the client,
6. **Client Rendering:** The browser receives the response and renders the webpage or displays the requested data to the user.

1.4 Browser and Web Server

A **web browser** is client-side software, like Chrome or Firefox, that requests, retrieves, and displays web content for a user,

A **web server** is hardware and software that stores, processes, and sends that web content to browsers upon request.

The browser initiates communication by sending an HTTP (or HTTPS) request to the server, which responds with the requested data, allowing the user to interact with the internet.

The Browser (Client)

A software application on your device (computer, phone, tablet).

Browser does:

- Fetches web pages, images, and other resources from web servers.
- Interprets and renders the content into a human-readable format for the user.
- Handles user interaction, such as clicking links or submitting forms.
- **Examples:** Google Chrome, Mozilla Firefox, Safari, Microsoft Edge.

Web Server (Server side)

- A computer system (hardware + software) that stores and delivers websites.
- Runs specialized software (like **Apache, Nginx, IIS**).
- Main functions:
 - Listens for incoming requests from browsers.
 - Processes requests (e.g., fetch a page, run scripts, query a database).
 - Sends responses back to the browser in the form of web pages.
 - Ensures security, load balancing, and uptime.

Examples of Web Server Software: Apache HTTP Server, Nginx, Microsoft IIS, LiteSpeed.

How They Work Together

1. User types a URL (e.g., **www.wu.edu.et**) into the **browser**.
2. The browser sends an **HTTP/HTTPS request** to the **web server** hosting that site.
3. The **web server** locates the requested resource (like index.html).
4. The web server sends the resource back as an **HTTP response**.
5. The **browser** interprets the HTML, CSS, and JavaScript, then displays the page to the user.

1.5 Document Types and MIME

Document Types

- A **document type** simply means the kind of file or resource (e.g., text, image, audio, video, PDF).
- Examples:
 - **Text documents** :- .txt, .docx, .pdf
 - **Web documents** :- .html, .css, .js
 - **Images** :- .jpg, .png, .gif
 - **Audio** :- .mp3, .wav
 - **Video** :- .mp4, .avi

These types tell us **what kind of content** a file contains.

MIME (Multipurpose Internet Mail Extensions) Types

- A **MIME type** is a standard way of indicating the nature and format of a file on the internet.
- Used in the **HTTP protocol** so that browsers know **how to handle a file**.
- MIME types are standardized two-part identifiers (like text/html or image/jpeg) that tell a browser or application the type and format of a document or file, enabling it to be processed or displayed correctly.
- A MIME type consists of a general type (e.g., text, image, application) and a specific subtype (e.g., html, jpeg, pdf), guiding how the data should be handled by the receiving client.
- Format:

type/subtype

type = general category (text, image, audio, video, application, multipart).

subtype = specific file format.

Common MIME Types

File Type	Extension	MIME Type
HTML	.html	text/html
CSS	.css	text/css
JavaScript	.js	application/javascript
JSON	.json	application/json
JPEG Image	.jpg	image/jpeg
PNG Image	.png	image/png
PDF	.pdf	application/pdf
MP3 Audio	.mp3	audio/mpeg
MP4 Video	.mp4	video/mp4

MIME types work

1. **Identification:** When a server sends a file to a browser, it includes a Content-Type header specifying the MIME type.
2. **Interpretation:** The browser then uses this information to understand the file's format.

3. **Action:** Based on the MIME type, the browser knows how to display or process the content, such as rendering HTML in the browser window, embedding a video, or prompting the user to download an attachment.

MIME types are important

- **Correct Handling:** They ensure that files are opened and displayed with the correct application or rendered properly by the browser.
- **Cross-Platform Compatibility:** MIME types provide a standardized way to handle diverse file types across different operating systems and devices.
- **Security:** Browsers use MIME types to safeguard users from harmful behaviors by correctly identifying file types.

1.6. Uniform Resource Locators(URLs)

A URL or Uniform Resource Locator is a unique identifier that is contained by all the resources available on the internet. It can help to locate a particular resource due to its uniqueness. It is also known as the web address. A URL consists of different parts like protocol, domain name, etc. The users can access the URLs by simply typing them inside the address bar or by clicking any button or link web page.

Structure of a URL

A URL starts with a protocol followed by the name of the resource that has to be accessed. URL uses the protocols as the primary access medium to access the domain or subdomain specified after that wherever the resource is located. It uses multiple protocols like

- **HTTP** (Hypertext Transfer Protocol),
- **HTTPS Protocol** (Secured HTTP),
- **FTP** (File Transfer Protocol)
- **TELNET** to access remote computers.

Mostly the protocol names are specified using the colons and the double forward slashes

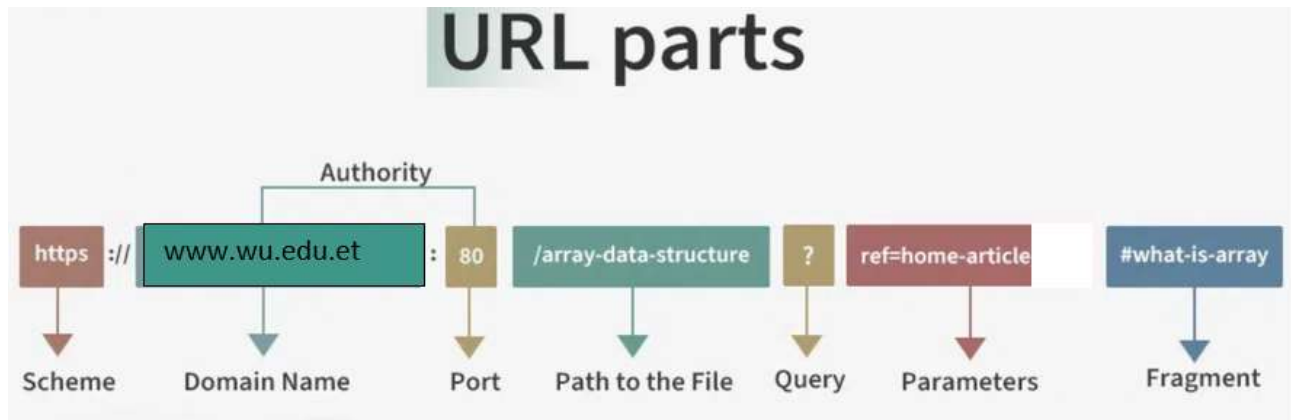
Optional Parts after the domain name in a URL:

- A path to a particular page or file can be specified.
- Some extra query parameters can also be specified.

- Network port to make the connection.
- Reference to a particular point in the file or a HTML element on the page.

Different Parts of a URL

A URL consists of multiple parts that can help you to visit a particular page on the internet. Every part of a URL has its own importance.



Reference URL:

<https://www.wu.edu.et/dsa/array-data-structure-guide/#what-is-array>

The protocol or scheme:

A URL starts with a protocol that is used to access the resource on the internet. The resource is accessed through the **Domain Name System** or DNS. There are multiple protocols available to use like **HTTP, HTTPS, FTP, TELNET** etc. The protocol used in the above URL is **https**.

Domain or Host Name:

It is the reference or name of the page that you are going to access on the internet. In this case, the domain name is: **www.wu.edu.et**.

Port Name:

A port on a website refers to a virtual network communication endpoint identified by a number, such as port 80 for HTTP or port 443 for HTTPS, that directs traffic to the correct application on a web server. These ports are essential for allowing computers and servers to communicate, ensuring that web pages, emails, and other data reach the intended service on a device.

It is defined just after the domain name by using the colons between itself and the domain name. Generally, it is not visible in the URL. The **domain name** and the **port name** combinely can be known as **Authority**. The default port for web services is **port 80 (:80)**.

Path:

It refers to the path or location of a particular file or page stored on the web server to access the content of it. The path used here is: **array-data-structure**.

Query:

A query mainly found in the dynamic pages. It consists of a **question mark(?)** followed by the parameters. In above URL query is: **?**.

Parameters:

These are the pieces of information inside a query string of URL. Multiple parameters can be passed to a URL by using the **ampersand(&)** symbol to separate them. The query parameter in above URL is: **ref=home-article**.

Fragments:

The fragments appear at the end of a URL starts with a **Hashtag(#)** symbol. These are the internal page references that refers to a specific section within the page. The fragment in the above URL is: **#what-is-array**.

1.7Domain Name System(DNS)

The Domain Name System (DNS) translates human-readable domain names (like www.example.com) into numerical IP addresses (like 192.168.2.1) that computers use to locate and connect with each other on the internet. It functions as the "phone book" of the internet, allowing users to access websites, services, and other network resources by remembering easy-to-use names instead of complex IP addresses.

Key Points about DNS:

1. Purpose:

- Converts domain names into IP addresses.
- Allows users to use easy names instead of remembering long numbers.

2. Hierarchy of DNS:

- **Root Domain:** The top of the DNS hierarchy (".").
- **Top-Level Domain (TLD):** Examples: .com, .org, .et.
- **Second-Level Domain:** Example: google in google.com.
- **Subdomains:** Example: mail.google.com.

3. DNS Components:

- **DNS Resolver:** The client (often your ISP or device) that queries DNS servers.
- **Root Server:** Directs queries to the correct TLD server.
- **TLD Server:** Directs to the authoritative server for that domain.
- **Authoritative DNS Server:** Holds the actual records (e.g., IP address).

DNS Works :-

1. **User Entry:** You type a domain name (e.g., **www.wu.edu.et**) into your web browser.
2. **Cache Check:** Your browser first checks its own local cache and the operating system's cache to see if it already knows the corresponding IP address.
3. **DNS Resolver Query:** If the IP address isn't in the cache, your system sends a request to a DNS resolver.
4. **Root Name Server:** The resolver starts by asking a root name server, which acts like an index to the internet's name system. The root server tells the resolver where to find the Top-Level Domain (TLD) (like .com) name server.
5. **TLD Name Server:** The resolver then contacts the appropriate TLD name server (e.g., the .com TLD server), which directs it to the correct authoritative name server for the domain.
6. **Authoritative Name Server:** This server has the definitive IP address for the domain name. It's the DNS server that **holds the actual records** (like the record with the IP address) for a domain. It provides the final, definitive answer to a DNS query for a specific domain by storing the most up-to-date information about that domain's IP addresses and other records
7. **Response:** The resolver receives the IP address from the authoritative server and sends it back to your operating system, which then gives it to the browser, allowing you to connect to the website.
8. **Caching:** The IP address is cached for future requests to speed up the process.

The domain **www.wu.edu.et** breaks down into the DNS hierarchy:

1. **Root server (.)**

- At the very top of the DNS tree is the **root**.
- The root servers don't hold the actual domain info; they just know where to find the **Top-Level Domain (TLD)** servers.

2. **Top-Level Domain (TLD)**

- In this case, the **TLD** is **.et** (Ethiopia's country-code top-level domain).
- The root server will direct the query to the **.et TLD servers**.

3. **Second-Level Domain (SLD)**

- Under .et, we have **.edu.et**, which is a **second-level domain** (usually reserved for educational institutions in Ethiopia).

4. **Third-Level Domain**

- Within .edu.et, we have **wu.edu.et**, which is the registered domain for a specific institution (**Wollo University**).
- Its authoritative DNS servers store the exact IP address mappings.

5. **Host/Subdomain**

- **www** is a **subdomain** of wu.edu.et.
- Often, **www** points to the main web server of that institution.

So, for **www.wu.edu.et**:

- **Root server** = . (the very top)
- **Top-Level Domain (TLD)** = .et

Illustrate the full query path (step by step, from your browser → root → .et → .edu.et → wu.edu.et → www)?

1.8TCP/ IP Protocols

TCP/IP, standing for Transmission Control Protocol/Internet Protocol, is the foundational suite of communication protocols for the internet and other computer networks. It defines the standards for how data is broken into packets, addressed, transmitted, routed, and received across networks,

enabling devices worldwide to communicate reliably. TCP handles reliable, ordered data delivery, while IP is responsible for addressing and routing the data packets.

The TCP/IP Model

The TCP/IP model is structured into four layers, each with specific functions:

- 1. Application Layer:** This is where applications interact with the network, handling tasks like web browsing, email, and file transfer.
- 2. Transport Layer:** Here, TCP manages the segmentation of data into packets, ensures reliable delivery, and reassembles them at the destination.
- 3. Internet (or Network) Layer:** IP is responsible for logical addressing (IP addresses) and routing these packets across different networks to reach their destination.
- 4. Link (or Network Access) Layer:** This layer handles the physical transmission of data over the network medium and the details of accessing the network.

Key Functions of TCP and IP

- **TCP (Transmission Control Protocol):**
 - **Reliable Delivery:** Guarantees that all data packets are delivered and received correctly.
 - **Ordered Delivery:** Ensures packets are reassembled in the correct sequence at the destination.
 - **Flow Control:** Manages the rate of data transmission to prevent overwhelming the receiver.
- **IP (Internet Protocol):**
 - **Addressing:** Assigns a unique IP address to each device on the network.
 - **Routing:** Determines the best path for data packets to travel across the network.
 - **Packet Handling:** Breaks down larger data into smaller packets for transmission.